

## Тройки върхове

Източните Кордилери са планинска верига в Андите, която се простира на територията на Боливия. Тя се състои от  $N$  планински върха, номерирани от 0 до  $N - 1$ . **Височината** на връх  $i$  ( $0 \leq i < N$ ) е  $H[i]$ , цяло число между 1 и  $N - 1$ , включително.

За всеки два върха  $i$  и  $j$ ,  $0 \leq i < j < N$ , **разстоянието** между тях е дефинирано като  $d(i, j) = j - i$ .

Според древни легенди на инките, тройка върхове е **митична**, ако изпълнява следното свойство: височините на трите върха **съответстват** на разстоянията им по двойки, **пренебрегвайки реда им**.

Формално, тройка индекси  $(i, j, k)$  е митична, ако

- $0 \leq i < j < k < N$  и
- височините  $(H[i], H[j], H[k])$  съвпадат с разстоянията по двойки  $(d(i, j), d(i, k), d(j, k))$ , пренебрегвайки реда. Например, за индексите 0, 1, 2 разстоянията по двойки са  $(1, 2, 1)$ , така че височините  $(H[0], H[1], H[2]) = (1, 1, 2)$ ,  $(H[0], H[1], H[2]) = (1, 2, 1)$  и  $(H[0], H[1], H[2]) = (2, 1, 1)$  им съответстват, но височините  $(H[0], H[1], H[2]) = (1, 2, 2)$  не им съответстват.

Тази задача се състои от две части като всяка подзадача е или от **Част I**, или от **Част II**. Можете да решавате подзадачите в произволен ред. По-специално, **не сте** задължени да решите цялата част I преди да започнете част II.

### Част I

При дадено описание на планинска верига, Вашата задача е да преброите броя митични тройки.

#### Детайли по имплементацията

Трябва да имплементирате следната функция.

```
long long count_triples(std::vector<int> H)
```

- $H$ : масив с дължина  $N$ , обозначаващ височините на върховете.
- Функцията ще бъде извикана точно веднъж на тест.

Функцията трябва да върне цяло число  $T$ , броя митични тройки в планинската верига.

## Ограничения

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$  за всяко  $i$ , спазващо  $0 \leq i < N$ .

## Подзадачи

Част I е за общо 70 точки.

| Подзадача | Точки | Допълнителни ограничения                                                                                                 |
|-----------|-------|--------------------------------------------------------------------------------------------------------------------------|
| 1         | 8     | $N \leq 100$                                                                                                             |
| 2         | 6     | $H[i] \leq 10$ за всяко $i$ , спазващо $0 \leq i < N$ .                                                                  |
| 3         | 10    | $N \leq 2000$                                                                                                            |
| 4         | 11    | Височините са подредени в ненамаляващ ред.<br>По-формално, $H[i - 1] \leq H[i]$ за всяко $i$ , спазващо $1 \leq i < N$ . |
| 5         | 16    | $N \leq 50\,000$                                                                                                         |
| 6         | 19    | Няма допълнителни ограничения.                                                                                           |

## Пример

Нека разгледаме следния пример.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Съществуват 3 митични тройки в планинската верига:

- За  $(i, j, k) = (1, 3, 4)$ , височините  $(1, 3, 2)$  съответстват на разстоянията по двойки  $(2, 3, 1)$ .
- За  $(i, j, k) = (2, 3, 6)$ , височините  $(4, 3, 1)$  съответстват на разстоянията по двойки  $(1, 4, 3)$ .
- За  $(i, j, k) = (3, 4, 6)$ , височините  $(3, 2, 1)$  съответстват на разстоянията по двойки  $(1, 3, 2)$ .

Следователно, функцията трябва да върне 3.

Забележете, че  $(0, 2, 4)$  не образуват митична тройка, понеже височините  $(4, 4, 2)$  не съответстват на разстоянията  $(2, 4, 2)$ .

## Част II

Вашата задача е да конструирате планински вериги с много митични тройки. Тази част се състои от 6 **output-only** подзадачи с **частично оценяване**.

Във всяка подзадача са дадени положителните цели числа  $M$  и  $K$  и трябва да конструирате планинска верига с **най-много**  $M$  върха. Ако Вашето решение съдържа **поне**  $K$  митични тройки, ще получите пълният резултат за тази подзадача. В противен случай, Вашият резултат ще бъде пропорционален на броя митични тройки, които се съдържат в решението Ви.

Забележете, че Вашето решение трябва да бъде валидна планинска верига. По-точно, нека решението Ви съдържа  $N$  върха ( $N$  трябва да отговаря на  $3 \leq N \leq M$ ). Тогава височината на връх  $i$  ( $0 \leq i < N$ ), обозначена като  $H[i]$ , трябва да бъде цяло число между 1 и  $N - 1$ , включително.

### Детайли по имплементацията

Съществуват два метода, за изпращане на решението Ви, като може да използвате, който и да е от двата за дадена подзадача:

- **Output file**
- **Procedure call**

За да предадете решението си чрез **output file**, създайте и предайте текстов файл в следния формат:

```
N
H[0] H[1] ... H[N-1]
```

За да предадете решението си чрез **procedure call**, трябва да имплементирате следната функция.

```
std::vector<int> construct_range(int M, int K)
```

- $M$ : максималният брой върхове.
- $K$ : желаният брой митични тройки.
- Тази функция ще бъде извикана точно веднъж на подзадача.

Функцията трябва да върне масив  $H$  с дължина  $N$ , задаващ височините на върховете.

## Подзадачи и оценяване

Част II е за общо 30 точки. За всяка подзадача, стойностите на  $M$  и  $K$  са фиксирани и дадени в следната таблица:

| Подзадача | Точки | $M$     | $K$        |
|-----------|-------|---------|------------|
| 7         | 5     | 20      | 30         |
| 8         | 5     | 500     | 2000       |
| 9         | 5     | 5000    | 50 000     |
| 10        | 5     | 30 000  | 700 000    |
| 11        | 5     | 100 000 | 2 000 000  |
| 12        | 5     | 200 000 | 12 000 000 |

За всяка подзадача, ако решението Ви не образува валидна планинска верига, резултатът Ви ще бъде 0 (съобщено като `Output isn't correct` в CMS).

Иначе, нека с  $T$  означим броят митични тройки във Вашето решение. Тогава Вашият резултат за подзадачата е:

$$5 \cdot \min \left( 1, \frac{T}{K} \right).$$

## Локален грейдър

Част I и II използват един и същ локален грейдър, като двете части се различават чрез първия ред на входа.

Формат на входа за част I:

```
1
N
H[0] H[1] ... H[N-1]
```

Формат на изхода за част I:

```
T
```

Формат на входа за част II:

```
2
M K
```

Формат на изхода за част II:

```
N
H[0] H[1] ... H[N-1]
```

Забележете, че изходът на локалния грейдър съвпада с необходимия формат за output file в част II.